KDD-Cup-99 Networking Dataset

Presented by Group 7

Sam Cordry, Liam Cummings, Dominic Vinciulla

Problem Scope & Dataset Background

PROBLEM DESCRIPTION

Network Intrusion Detection

Using different machine learning algorithms, our goal is to build a system that can **detect malicious or dangerous network traffic** within a system.

Primary Objective:

Developing robust automated classifiers for system security monitoring.

DATASET BACKGROUND

The KDD-Cup-99 Challenge

Massive Volume
Very large dataset requiring efficient processing.

High Dimensionality
Very-high dimensional featureset.

Class Imbalance
Strongly imbalanced classes necessitating model combat strategies.

Project Hypothesis & Visualization Strategy

MODEL HYPOTHESIS

K-Nearest & SVM

We believe these will be the **most accurate models** for forming conclusions about our networking data.

Key Prediction:

Superior handling of high-dimensional feature sets compared to baseline algorithms.

VISUALIZATION PLAN

Evidence-Based Analysis

Confusion Matrices

- Visualizing classification performance across all classes.

Metric Printouts

- Evaluating precision, recall, and F1-scores for hyperparameter tuning.

Hyperparameter Mapping

- Showing how model tweaks correlate with data compatibility.

Initial Dataset and Preprocessing

Dataset Scope

Initial Shape: (4,898,431, 110)

Preprocessing Steps

- Binary Target Encoding
  0 = Normal, 1 = Anomaly
- Missing Value Replacement
  Mean for numerical, Constant for categorical
- One-Hot Encoding
  Applied to all categorical features
- Z-Score Standardization
  Standardized all numeric features

Dimensionality Reduction (PCA)

95% Variance

Maintained original dataset integrity

Resulting Shape:

(4,898,431, 19)

Reduced dimensionality by ~82.7%

Model Training - Implementation

cuML - GPU Accelerated Training

cuML is an open-source NVIDIA Python library that implements the sklearn API to allow for zero code-change implementation of GPU acceleration. This was used to help train the SVM and KNN models due to their complexity.

Reduced training & tuning by estimated factor of 1,200+ for SVM (RTX 4070 Laptop)

Scikit-learn - Testing and CPU Training

The program defaults to regular sklearn if cuML or a compatible GPU is unavailable.

Key sklearn classes used:
SVC, IsolationForest, and KNeighborsClassifier

Support Vector Machine - Hyperparameters

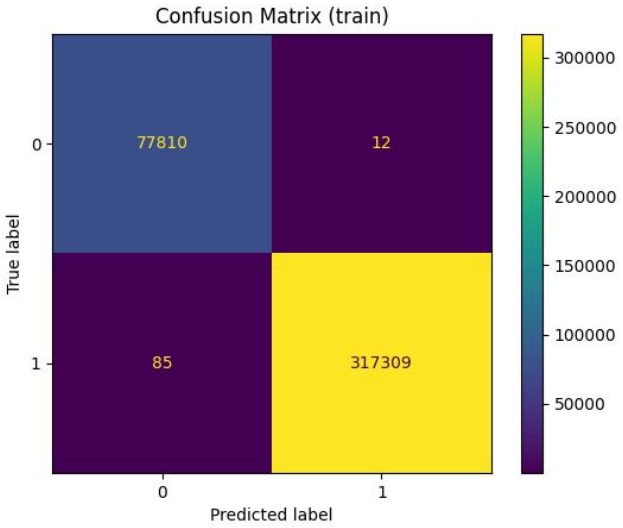
F1 Heatmap

Python Implementation

```python
param_grid = {
    'C': np.logspace(-3, 3, 10),
    'gamma': np.logspace(-4, 1, 10)
}

search = GridSearchCV(
    estimator=base_model,
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=1 if GPU else -1,
    verbose=2
)

search.fit(X_train, y_train)
```

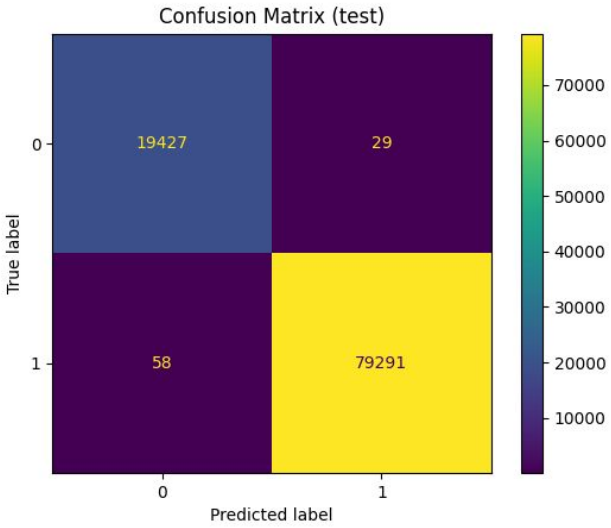
Support Vector Machine - Performance

Confusion Matrix: Training Data



Training Accuracy: 0.99754

Confusion Matrix: Testing Data



Testing Accuracy: 0.999605

K-Nearest - Hyperparameters

We tested multiple values of K on a 10% subset of the dataset to find the optimal balance between accuracy and generalization.

Accuracy by K-Value

K = 1 →	0.99951419
K = 3 →	**0.99950407**
K = 5 →	0.99946358
K = 7 →	0.99943322
K = 9 →	0.99940286

Selected: K = 3

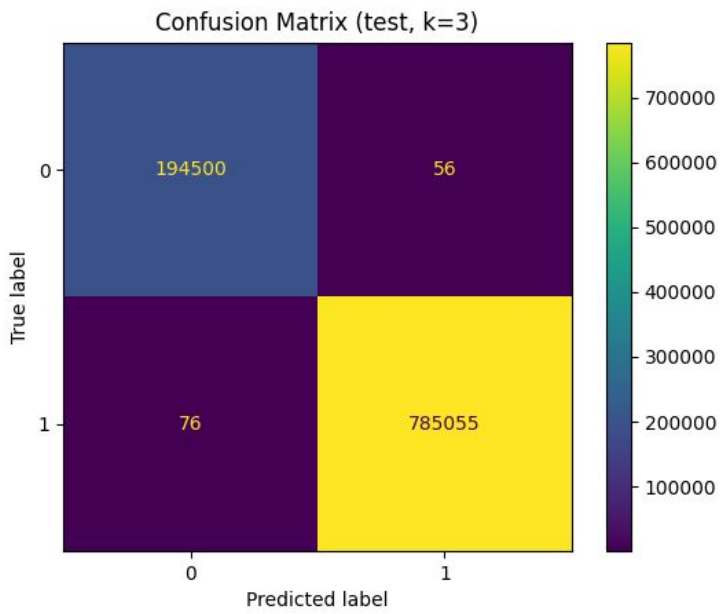
Decision Rationale:

- While **K = 1** showed the highest numerical accuracy, it is highly susceptible to noise.
- We chose **K = 3** to avoid potential overfitting of the training data and improve model robustness.

Hyperparameter tuning conducted using 10% subset of KDD-Cup-99 dataset.

K-Nearest (K = 3) Performance

Performance evaluation of the K-Nearest Neighbors model on the full KDD-Cup-99 testing dataset.



Final Test Accuracy

0.999865

Observation

The near-perfect accuracy strongly suggests potential **overfitting** issues:

- High dependency on specific data patterns
- Potential lack of generalization to unseen structures

Random Isolation Forest - Hyperparameters

The following hyperparameters were found to give the most consistently accurate models:

Number of Estimators

100

Contamination

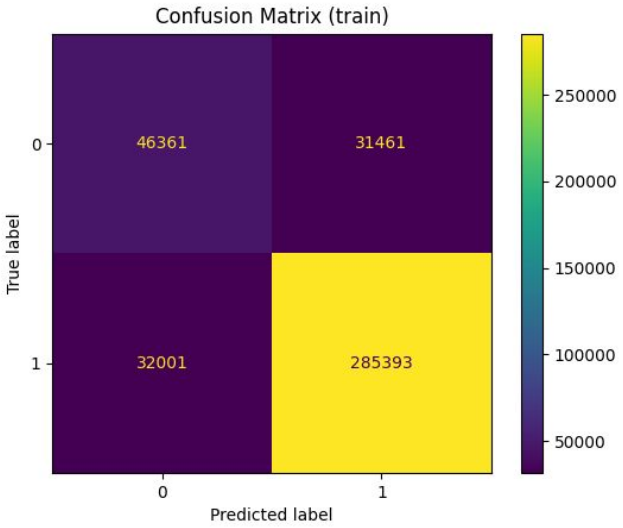
0.2

Testing Ranges

- **Number of Estimators:** Tested on the range 1-1000
- **Contamination:** Tested on the range 0-1

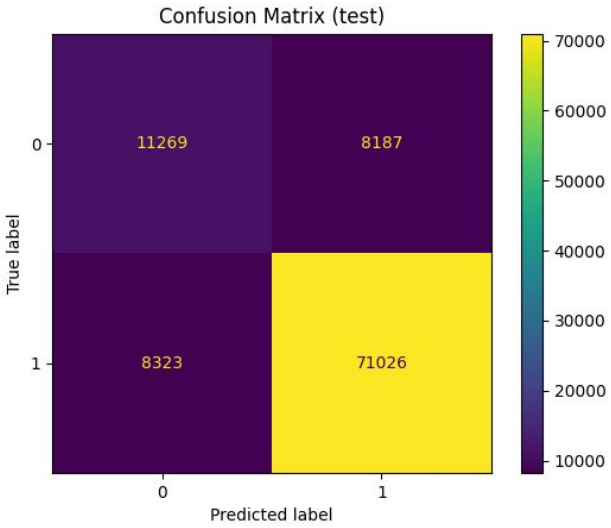
Random Isolation Forest - Performance

Confusion Matrix: Training Data



Training Accuracy: 0.839

Confusion Matrix: Testing Data



Testing Accuracy: 0.833

Random Isolation Forest - Classification Reports

Classification Report: Training Data

```
Classification Report:
              precision    recall  f1-score   support

          -1       0.59      0.60      0.59     77822
           1       0.90      0.90      0.90    317394

    accuracy                           0.84    395216
   macro avg       0.75      0.75      0.75    395216
weighted avg       0.84      0.84      0.84    395216
```

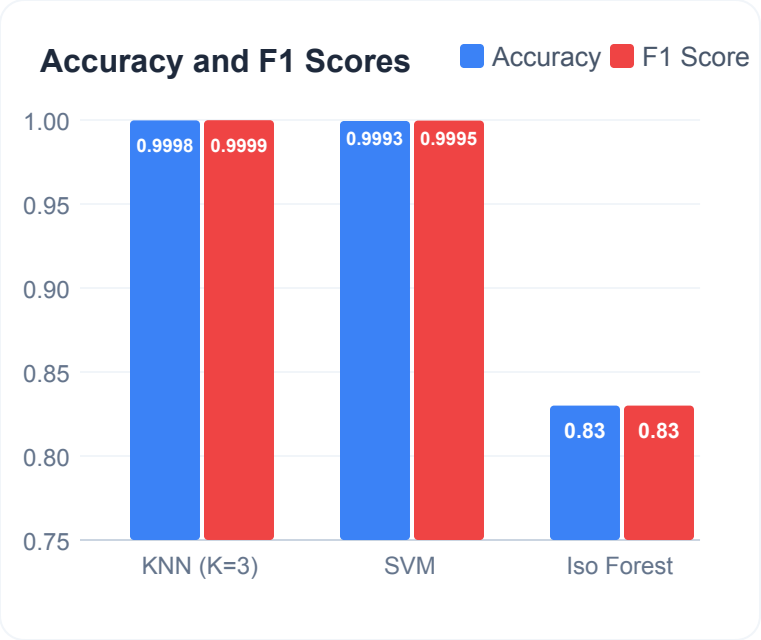
Classification Report: Testing Data

```
Classification Report:
              precision    recall  f1-score   support

          -1       0.58      0.58      0.58     19456
           1       0.90      0.90      0.90     79349

    accuracy                           0.83     98805
   macro avg       0.74      0.74      0.74     98805
weighted avg       0.83      0.83      0.83     98805
```

Model Comparison: Accuracy vs. F1 Score

KNN achieved the best overall performance, while SVM offers comparable accuracy. Isolation Forest shows a significant performance gap.

Model	Accuracy	Precision	Recall	F1 Score
KNN (K=3)	0.99987	0.999928	0.999903	0.999916
SVM	0.99932	0.99977	0.99938	0.99958
Isolation Forest	0.83	0.83	0.83	0.83



Future Ideas/Work

If we had more time for the project, there are some changes we would likely make to potentially improve the performance of our models:

Introduction of SMOTE

As the kdd-cup dataset it notoriously imbalanced, we could introduce SMOTE algorithms to help introduce more balance to our classes, and potentially improve our performance.

Conclusion

KNN (Best Performance)

KNN achieved the highest performance across all metrics, capturing patterns within the KDD dataset very well.

Note: Near-perfect accuracy may suggest potential overfitting or dataset dependency.

SVM (Good Generalization)

SVM generalized well to the data, showing comparable capability to KNN.

Note: Similar risk of overfitting or data leakage due to unusually high accuracy.

Random Isolation Forest

Performed measurably worse than the other two algorithms. The data structure is not easily split via isolation.

Note: Performance might improve with the introduction of SMOTE algorithms.